

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 5
CLEAN CODE & DEFENSIVE PROGRAMMING**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iii
BAB I. HASIL PRAKTIKUM	4
I.1 Meaningfull Names	4
I.1.A. Source Code	4
I.1.B. Analisa	4
I.2 Use Pronounceable Names	5
I.2.A. Source code	5
I.2.B. Analisa	6
I.3 Switch Statement	6
I.3.A. Source code	6
I.3.B. Analisa	7
I.4 Data Abstraction	8
I.4.A. Source code	8
I.4.B. Analisa	8
I.5 Comments	9
I.5.A. Source code	9
I.5.B. Analisa	11
I.6 Formatting	11
I.6.A. Source code	11
I.6.B. Analisa	13
I.7 Assertion	13
I.7.A. Source code	13
I.7.B. Hasil	14
I.7.C. Analisa	14

I.8 Single Catch Block	15
I.8.A. Source code	15
I.8.B. Hasil	15
I.8.C. Analisa	15
I.9 Multiple Catch Block	16
I.9.A. Source code	16
I.9.B. Analisa	16
I.10 The Finally Block	17
I.10.A. Source code	17
I.10.B. Hasil	17
I.10.C. Analisa	18
BAB II. TUGAS PRAKTIKUM	19
II.1 Clean Code	19
II.1.A. Source code	19
II.1.B. Analisis	20
II.2 Defensive Programming	21
II.2.A. Source code	21
II.2.B. Analisis	23
BAB III. KESIMPULAN	24

DAFTAR GAMBAR

gambar 1. 1 Hasil assertion	14
gambar 1. 2 Hasil Finally Block	17

BAB I. HASIL PRAKTIKUM

I.1 Meaningfull Names

I.1.A. Source Code

A. Sebelum

```
public List<int[]> getThem() {  
  
    List<int[]> list1 = new ArrayList<int[]>();  
  
    for (int[] x : theList)  
  
        if (x[0] == 4)  
  
            list1.add(x);  
  
    return list1;  
  
}
```

B. Sesudah

```
public final static int STATUS_VALUE = 0;  
  
public final static int FLAGGED = 4;  
  
public List<int[]> getFlaggedCells() {  
  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
  
    for (int[] cell : gameBoard)  
  
        if (cell[STATUS_VALUE] == FLAGGED)  
  
            flaggedCells.add(cell);  
  
    return flaggedCells;  
  
}
```

I.1.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") menunjukkan pelanggaran berat terhadap prinsip Meaningful Names. Penggunaan nama variabel yang ambigu seperti list1, theList, dan x sama sekali tidak mengungkapkan niat penulis (intention-revealing), sehingga pembaca harus menebak-nebak apa fungsi kode tersebut. Selain itu, kode ini mengandung Magic Numbers (angka 0 dan 4) tanpa konteks yang jelas. Seorang pengembang lain yang membaca kode ini tidak akan

mengerti mengapa elemen ke-0 yang dicek atau apa arti dari nilai 4, yang membuat kode ini sangat sulit dipelihara (unmaintainable) dan rentan terhadap bug jika logika bisnis berubah.

Sebaliknya, kode bagian kedua ("Sesudah Clean Code") telah berhasil mentransformasi kode menjadi narasi yang dapat dibaca manusia (human-readable). Dengan mengganti angka mentah menjadi konstanta bernama STATUS_VALUE dan FLAGGED, serta mengubah nama metode menjadi getFlaggedCells, kode ini menjadi self-documenting (mendokumentasikan dirinya sendiri). Kini terlihat jelas bahwa konteksnya adalah sebuah papan permainan (gameBoard) di mana sistem mencari sel-sel yang ditandai (flagged). Perubahan ini secara drastis menurunkan beban kognitif pembaca dan meningkatkan ketahanan kode, karena definisi "status" dan "flagged" kini terpusat dan mudah dipahami.

I.2 Use Pronounceable Names

I.2.A. Source code

A. Sebelum

```
class DtaRcrd102 {  
  
    private Date genymdhms;  
  
    private Date modymdhms;  
  
    private final String pszqint = "102";  
  
    /* ... */  
  
}
```

B. Sesudah

```
class Customer {  
  
    private Date generationTimestamp;  
  
    private Date modificationTimestamp;  
  
    private final String recordId = "102";  
  
    /* ... */  
  
}
```

I.2.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") menyoroti masalah serius terkait prinsip Use Pronounceable Names, di mana pengembang menggunakan singkatan kriptik dan kombinasi karakter yang hampir mustahil untuk dilafalkan secara lisan. Variabel seperti `genymdhms` (singkatan dari `generation year-month-day...`) dan `pszqint` memaksa otak pembaca untuk melakukan "dekoding" mental setiap kali melihatnya, yang meningkatkan beban kognitif secara signifikan. Selain sulit dipahami, penamaan semacam ini sangat menghambat komunikasi tim; bayangkan betapa canggung dan tidak efisiennya jika pengembang harus mendiskusikan variabel `DtaRcrd102` atau `modymdhms` dalam sebuah rapat, di mana mereka terpaksa harus mengeja nama variabel tersebut huruf demi huruf agar tidak terjadi kesalahpahaman.

Sebaliknya, kode versi "Sesudah Clean Code" menunjukkan perbaikan drastis dengan menggunakan kata-kata bahasa Inggris yang utuh dan wajar. Transformasi dari `DtaRcrd102` menjadi `Customer` dan `genymdhms` menjadi `generationTimestamp` menjadikan kode tersebut dapat dibaca dan diucapkan selayaknya bahasa manusia alami. Perubahan ini tidak hanya membuat kode lebih "bersih" secara visual, tetapi juga mendukung Defensive Programming dengan mengurangi ambiguitas; pengembang lain kini dapat langsung memahami bahwa variabel tersebut menyimpan waktu pembuatan (timestamp) tanpa perlu menebak-nebak format singkatan, sehingga memperlancar alur kerja kolaboratif dan pemeliharaan kode jangka panjang.

I.3 Switch Statement

I.3.A. Source code

A. Sebelum

```
int payAmount() {
    switch (getType()) {
        case EmployeeType.ENGINEER:
            return _monthlySalary;
        case EmployeeType.SALESMAN:
            return _monthlySalary + _commission;
        case EmployeeType.MANAGER:
            return _monthlySalary + _bonus;
        default:
            throw new Exception("Incorrect Employee");
    }
}
```

```
}  
}
```

B. Sesudah

```
class EmployeeType {  
    abstract int payAmount(Employee emp);  
}  
class Salesman {  
    int payAmount(Employee emp) {  
        return emp.getMonthlySalary() + emp.getCommission();  
    }  
}  
class Manager {  
    int payAmount(Employee emp) {  
        return emp.getMonthlySalary() + emp.getBonus();  
    }  
}
```

I.3.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") mengidentifikasi code smell klasik di mana pernyataan switch digunakan untuk memilih perilaku berdasarkan tipe objek. Pendekatan prosedural ini melanggar Open/Closed Principle (OCP), karena fungsi `payAmount` tidak "tertutup" terhadap modifikasi; setiap kali ada penambahan tipe karyawan baru (misalnya "Intern"), pengembang dipaksa untuk membongkar dan mengubah kode yang sudah berjalan dan teruji. Hal ini sangat berisiko dalam konteks Defensive Programming, karena jika logika switch serupa tersebar di banyak tempat dalam aplikasi, risiko lupa memperbarui salah satu bagian akan menyebabkan bug inkonsistensi yang fatal.

Sebaliknya, kode bagian kedua ("Sesudah Clean Code") memecahkan masalah ini dengan teknik refactoring *Replace Conditional with Polymorphism*. Dengan mendistribusikan logika perhitungan gaji ke dalam metode `payAmount` di masing-masing kelas turunan (`Salesman`, `Manager`), kode kini mematuhi *Single Responsibility Principle* (SRP). Desain ini jauh lebih defensif dan robust: penambahan tipe karyawan baru hanya memerlukan pembuatan kelas baru tanpa menyentuh kode lama, dan mekanisme pewarisan (*inheritance*) menjamin bahwa

setiap tipe karyawan pasti memiliki implementasi pembayaran gaji yang valid, menghilangkan ketergantungan pada blok default yang rentan error.

I.4 Data Abstraction

I.4.A. Source code

A. Sebelum

```
package Latihan.DataAbstraction;

public class Point {
    public double x;
    public double y;
}
```

B. Sesudah

```
package Latihan.DataAbstraction;

public interface Point1 {
    double getX();
    double getY();
    void setCartesian(double x, double y);
    double getR();
    double getTheta();
    void setPolar(double r, double theta);
}
```

I.4.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") memperlihatkan bentuk struktur data yang telanjang (concrete point), di mana variabel `x` dan `y` dideklarasikan sebagai `public`. Ini melanggar prinsip Data Abstraction karena kelas tersebut mengekspos detail implementasinya secara gamblang kepada dunia luar. Pengguna kelas dipaksa untuk bergantung pada fakta bahwa koordinat disimpan dalam format Kartesius. Tidak ada enkapsulasi atau penyembunyian informasi di sini; jika di masa depan pengembang ingin mengubah cara penyimpanan data internal menjadi koordinat Polar, semua kode klien yang mengakses variabel `.x` dan `.y` secara langsung akan rusak (breaking changes), menjadikan kode ini sangat rapuh dan sulit dipelihara.

Sebaliknya, kode bagian kedua ("Sesudah Clean Code") menerapkan abstraksi data yang sesungguhnya melalui penggunaan interface. Alih-alih mengekspos variabel, kode ini mengekspos perilaku (behavior) abstrak melalui metode seperti `getX()`, `getY()`, `getR()`, dan `getTheta()`. Pengguna antarmuka ini tidak mengetahui—dan tidak perlu peduli—apakah data di dalamnya sebenarnya disimpan sebagai koordinat Kartesius atau Polar; mereka hanya berinteraksi melalui kontrak metode yang tersedia. Pendekatan ini merupakan bentuk Defensive Programming yang baik karena menciptakan dinding pemisah antara definisi data dan implementasinya, memberikan kebebasan kepada pengembang untuk memodifikasi logika internal tanpa risiko merusak kode lain yang menggunakan objek tersebut.

I.5 Comments

I.5.A. Source code

A. Bad Comments

```
package Latihan.Comments;

public class BadComments {
    //Journal comment
    /**
     * Changes (from 11-Oct-2001)
     * -----
     * 11-Oct-2001 : Re-organised the class and moved it to new
     * package com.jrefinery.date (DG);
     * 05-Nov-2001 : Added a getDescription() method, and
     * eliminated NotableDate class (DG);
     * 12-Nov-2001 : IBD requires setDescription() method, now
     * that NotableDate class is gone (DG); Changed
     * getPreviousDayOfWeek(),
     * getFollowingDayOfWeek() and
     * getNearestDayOfWeek() to correct bugs (DG);
     * 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
     * 29-May-2002 : Moved the month constants into a separate
     * interface (MonthConstants) (DG);
     */

    /**
     * Default constructor.
     */
}
```

```

protected void AnnualDateRule() {}

/** The day of the month. */
private int dayOfMonth;

/**
 * Returns the day of the month.
 * @return the day of the month.
 * */

public int getDayOfMonth() {
    return dayOfMonth;
}
}

```

B. Good Comments

```

package Latihan.Comments;

import java.util.regex.Pattern;

public class GoodComments {
    class Responder{}

    //legal comments
    // Copyright (C) 2011 by Osoco. All rights reserved.
    // Released under the terms of the GNU General Public License
    // version 2 or later.

    //informative comments
    // Returns an instance of the Responder being tested.
    protected Responder responderInstance() {
        return null;
    }

    // renaming the function: responderBeingTested
    // format matched kk:mm:ss EEE, MMM dd, yyyy
    Pattern timeMatcher = Pattern.compile("\\d*:\\d*:\\d* \\w*, \\w*
\\d*,\\d*");

    //clarification
    /*
    void assertTrue(a.compareTo(b) == -1); // a < b
    */
}

```

```
void assertTrue(b.compareTo(a) == 1); // b > a

*/
}
```

I.5.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") menyoroti penggunaan komentar yang redundant (berlebih) dan usang, khususnya keberadaan Journal Comments dan Noise Comments. Blok komentar panjang yang mencatat riwayat perubahan file (tanggal, penulis, deskripsi perubahan) sepenuhnya tidak diperlukan lagi karena tugas tersebut kini sudah ditangani jauh lebih efisien oleh Version Control System (seperti Git). Selain itu, komentar Javadoc seperti "Default constructor" atau "Returns the day of the month" hanya mengulang apa yang sudah jelas tertulis dalam nama metode dan variabel. Hal ini melanggar prinsip Clean Code karena hanya menambah "kebisingan" visual tanpa memberikan wawasan baru, yang justru mengalihkan fokus pengembang dari logika kode yang sebenarnya.

Sebaliknya, kode bagian kedua ("Sesudah Clean Code") mendemonstrasikan penggunaan komentar yang memiliki nilai fungsional dan defensive. Komentar kini digunakan secara strategis untuk hal-hal yang tidak bisa dijelaskan oleh kode itu sendiri, seperti Legal Comments untuk lisensi hak cipta yang wajib ada, serta komentar informatif yang menjelaskan pola Regular Expression (Regex) yang kompleks dan sulit dibaca secara sekilas. Selain itu, komentar klarifikasi pada logika perbandingan (menjelaskan bahwa hasil -1 berarti $a < b$) sangat krusial untuk mencegah kesalahpahaman logika oleh pengembang lain. Ini adalah penerapan yang tepat di mana komentar berfungsi untuk memperjelas intent (niat) dan kompleksitas, bukan sekadar menduplikasi kode.

I.6 Formatting

I.6.A. Source code

A. Sebelum

```
package Latihan.Formatting;

import com.sun.net.httpserver.Request;
```

```

import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class FitNesseExpediter implements ResponseSender {
    private Socket socket;
    private InputStream input;
    private OutputStream output;
    private Request request;
    private Respose response;
    private FitNesseContext context;
    protected long requestParsingTimeLimit;
    protected long responseParsingDeadLine;
    private boolean hasError;
}

```

B. Sesudah

```

package Latihan.Formatting;

import com.sun.net.httpserver.Request;

import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class FitNesseExpediter implements ResponseSender {
    private Socket socket;
    private InputStream input;
    private OutputStream output;
    private Request request;
    private Respose response;
    private FitNesseContext context;
    protected long requestParsingTimeLimit;
    private long requestProgress;
    private long requestParsingDeadline;
    private boolean hasError;
}

```

I.6.B. Analisa

Analisis pada kode bagian pertama ("Sebelum Clean Code") menunjukkan pengabaian terhadap standar Vertical Formatting dan indentasi yang baik. Ketidakkonsistenan spasi pada deklarasi variabel—di mana baris socket memiliki indentasi sementara baris-baris berikutnya seperti input dan output rata kiri—menciptakan gangguan visual yang merusak hierarki kode. Dalam prinsip Clean Code, format yang buruk bukan sekadar masalah estetika; kekacauan visual ini menyulitkan mata untuk memindai struktur kelas dengan cepat dan meningkatkan risiko kesalahan interpretasi cakupan (scope) variabel. Hal ini secara tidak langsung bertentangan dengan semangat Defensive Programming yang bertujuan meminimalkan potensi human error akibat kebingungan membaca kode.

Sebaliknya, kode bagian kedua ("Sesudah Clean Code") mendemonstrasikan penerapan tata letak yang disiplin dan konsisten. Seluruh atribut kelas kini sejajar secara vertikal dengan tingkat indentasi yang seragam, menciptakan aliran visual yang tenang, terorganisir, dan profesional. Perbaikan format ini secara signifikan meningkatkan keterbacaan (readability), memudahkan navigasi kode, dan memperjelas hubungan hierarkis bahwa semua variabel tersebut adalah anggota sejajar dari kelas FitNesseExpediter. Konsistensi gaya ini sangat krusial dalam kerja tim, karena memastikan kode tetap mudah dipelihara (maintainable) tanpa membebani pengembang lain dengan gaya penulisan yang acak-acakan.

I.7 Assertion

I.7.A. Source code

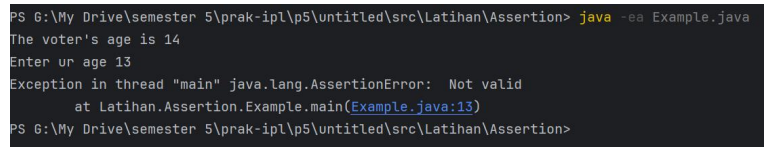
```
package Latihan.Assertion;

import java.util.Scanner;

public class Example {
    public static void main(String[] args) {
        int age = 14;
        assert age <= 18 : "Cannot Vote";
        System.out.println("The voter's age is " + age);
        Scanner scanner = new Scanner( System.in );
        System.out.print("Enter ur age ");
        int value = scanner.nextInt();
        assert value>=18:" Not valid";
    }
}
```

```
        System.out.println("value is "+value);  
    }  
}
```

I.7.B. Hasil



```
PS G:\My Drive\semester 5\prak-ipl\p5\untitled\src\Latihan\Assertion> java -ea Example.java  
The voter's age is 14  
Enter ur age 13  
Exception in thread "main" java.lang.AssertionError: Not valid  
    at Latihan.Assertion.Example.main(Example.java:13)  
PS G:\My Drive\semester 5\prak-ipl\p5\untitled\src\Latihan\Assertion>
```

gambar 1. 1 Hasil assertion

I.7.C. Analisa

Analisis kode ini mendemonstrasikan penerapan teknik Defensive Programming menggunakan mekanisme assert di Java, yang bertujuan untuk memvalidasi asumsi-asumsi programmer selama fase pengembangan. Dengan menjalankan program menggunakan flag -ea (enable assertions), mesin virtual Java (JVM) diinstruksikan untuk mengevaluasi pernyataan assert. Jika kondisi boolean yang diuji bernilai false (misalnya pada baris assert value ≥ 18), sistem akan menerapkan prinsip Fail Fast dengan segera menghentikan eksekusi dan melempar AssertionError. Teknik ini sangat efektif untuk menangkap bug logika atau keadaan sistem yang tidak valid sedini mungkin sebelum menyebabkan kerusakan data yang lebih parah di bagian kode selanjutnya.

Dari sudut pandang Clean Code, penggunaan assertion membuat kode lebih ringkas dan ekspresif dibandingkan memenjara kode dalam blok if-else yang rumit hanya untuk pengecekan kondisi internal. Pernyataan assert age ≤ 18 : "Cannot Vote" (meskipun logika bisnisnya agak ambigu di sini) secara eksplisit mendokumentasikan invariabel atau "kebenaran mutlak" yang diharapkan oleh sistem pada titik tersebut. Namun, perlu dicatat sebagai praktik defensif yang baik, validasi input pengguna (seperti hasil scanner.nextInt()) idealnya ditangani dengan Exception Handling standar, karena assertion dapat dinonaktifkan di lingkungan produksi, yang berisiko meloloskan data tidak valid jika administrator lupa mengaktifkan flag -ea.

I.8 Single Catch Block

I.8.A. Source code

```
package Latihan.SingleCatchblock;

public class ExceptTest {
    public static void main(String[] args) {
        try {
            int a[] = new int[2];
            System.out.println("Access element three :" + a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception thrown :" + e);
        }
        System.out.println("Out of the block");
    }
}
```

I.8.B. Hasil

I.8.C. Analisa

Analisis pada kelas ExceptTest mendemonstrasikan penerapan teknik Exception Handling sebagai pilar utama Defensive Programming. Kode ini secara proaktif mengantisipasi potensi kegagalan saat mengakses elemen array di luar batas yang dialokasikan (indeks 3 pada array berukuran 2). Dengan membungkus operasi berisiko tersebut dalam blok try, program mencegah terjadinya penghentian paksa (abnormal termination) yang dapat merusak pengalaman pengguna atau integritas data. Penggunaan blok catch yang spesifik menangkap ArrayIndexOutOfBoundsException menunjukkan praktik yang baik dibandingkan menangkap Exception generik, karena memungkinkan penanganan kesalahan yang lebih terarah dan presisi sesuai jenis masalah yang terjadi.

Dari perspektif stabilitas sistem, mekanisme ini menjamin keberlangsungan eksekusi program (program continuity). Ketika pengecualian terjadi, alur kontrol dialihkan ke blok catch untuk mencatat pesan kesalahan, lalu program dapat pulih dan melanjutkan eksekusi instruksi selanjutnya ("Out of the block"). Ini adalah implementasi konsep Graceful Degradation, di mana sistem tetap beroperasi meskipun terjadi kesalahan pada komponen tertentu, alih-alih mengalami crash total. Hal ini menjadikan kode lebih robust (tangguh) dan clean karena memisahkan logika bisnis utama dari logika penanganan kesalahan secara eksplisit.

I.9 Multiple Catch Block

I.9.A. Source code

```
package Latihan.MultipleCatchBlock;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class ExceptTest {

    public static void main(String[] args) {
        try {
            file = new FileInputStream(fileName);
            x = (byte) file.read();
        } catch (IOException i) {
            i.printStackTrace();
            return -1;
        } catch (FileNotFoundException f) // Not valid! {
            f.printStackTrace();
            return -1;
        }
    }
}
```

I.9.B. Analisa

Analisis pada kode ExceptTest menunjukkan kesalahan fundamental dalam urutan penanganan eksepsi (Exception Handling Hierarchy). FileNotFoundException secara teknis merupakan subclass (turunan) dari IOException. Karena blok catch untuk IOException (parent) diletakkan lebih awal, ia akan menangkap semua jenis kesalahan input/output termasuk kasus file tidak ditemukan. Akibatnya, blok catch kedua untuk FileNotFoundException menjadi kode yang tidak pernah bisa dijangkau (unreachable code). Hal ini menyebabkan compile-time error. Dalam konteks Defensive Programming, kesalahan urutan ini fatal karena menghalangi program untuk mengenali dan menangani kesalahan spesifik secara akurat, memaksa semua error dianggap sebagai kesalahan IO umum.

Untuk memperbaiki kode ini agar mematuhi prinsip Clean Code dan menjadi defensif, urutan blok catch harus dibalik: eksepsi yang paling spesifik harus ditangani terlebih dahulu sebelum eksepsi yang lebih umum (most specific to most general). Seharusnya catch (`FileNotFoundException f`) ditempatkan di atas catch (`IOException i`). Dengan urutan yang benar, program dapat membedakan respons mitigasi—misalnya, meminta pengguna mengecek ulang nama file jika `FileNotFoundException` terjadi, versus memeriksa koneksi disk jika `IOException` lain terjadi. Strategi ini memastikan aplikasi tidak hanya berjalan ("catch-all"), tetapi juga mampu memberikan penanganan kesalahan yang cerdas dan relevan sesuai konteks masalahnya.

I.10 The Finally Block

I.10.A. Source code

```
package Latihan.FinallyBlock;

public class ExceptTest {
    public static void main(String args[]) {
        int a[] = new int[2];
        try {
            System.out.println("Access element three :" + a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception thrown :" + e);
        } finally {
            a[0] = 6;
            System.out.println("First element value: " + a[0]);
            System.out.println("The finally statement is executed");
        }
    }
}
```

I.10.B. Hasil

```
Exception thrown :java.lang.ArrayIndexOutOfBoundsException:
    Index 3 out of bounds for length 2
First element value: 6
The finally statement is executed

Process finished with exit code 0
```

gambar 1. 2 Hasil Finally Block

I.10.C. Analisa

Analisis pada kode ExceptTest mendemonstrasikan peran vital blok finally dalam mekanisme Defensive Programming untuk menjamin eksekusi kode yang deterministik. Kode ini menunjukkan bahwa meskipun terjadi kegagalan fatal pada blok try akibat akses indeks array di luar batas (yang memicu `ArrayIndexOutOfBoundsException`), instruksi di dalam blok finally tetap dijalankan tanpa kecuali setelah blok catch. Mekanisme ini membuktikan bahwa blok finally bertindak sebagai "jaring pengaman" terakhir yang memprioritaskan penyelesaian tugas-tugas kritis—seperti penetapan nilai `a[0] = 6` dan pencetakan log—terlepas dari apakah alur program berjalan sukses atau mengalami crash di tengah jalan.

Dari perspektif Clean Code dan reliabilitas sistem, pola ini adalah fondasi standar untuk Manajemen Sumber Daya (Resource Management). Meskipun contoh di atas hanya memanipulasi variabel sederhana, dalam skenario nyata, struktur ini wajib digunakan untuk operasi pembersihan (cleanup), seperti menutup koneksi database, memutus socket jaringan, atau menutup file streams. Dengan menempatkan kode terminasi di dalam finally, pengembang menerapkan pertahanan berlapis untuk mencegah kebocoran memori (memory leaks) atau penguncian sumber daya (deadlocks), memastikan aplikasi selalu meninggalkan keadaan sistem yang bersih dan stabil sebelum selesai dieksekusi.

BAB II. TUGAS PRAKTIKUM

II.1 Clean Code

II.1.A. Source code

A. Naming

```
package Tugas.CleanCode;

public class kelasPengguna {
    /*
    protected String n;
    protected String p;

    */

    protected String username;
    protected String password;

    /*
    public void penyimpanan(String n,String p){
        //isi code
    }

    */

    public void penyimpanan(String username,String password){
        //isi code
    }
}
```

B. Formatting

```
package Tugas.CleanCode;

public class Formatting {
    /*
    fill(80,204,55);
    rect(100,0,200,400);
    var draw = function(){
        var numCircles = 6;
```

```

        for (var i=numCircles; i>1; i--){
            var radius=i*7;
            if (mouseX>100&&mouseX<300) {
                fill(255,255,255);
            }else{
                fill(80,204,55);
            }
            ellipse(mouseX,mouseY,radius,radius);}};

    */

    /*
    fill(80,204,55);rect(100,0,200,400);
    var draw=function(){var numCircles = 6;for (var i=numCircles; i>1;
i--){var
    radius=i*7;
    if (mouseX>100&&mouseX<300)

{fill(255,255,255);}else{fill(80,204,55);}ellipse(mouseX,mouseY,radiu
s,radi
    us);}};
    */
}

```

II.1.B. Analisis

Analisis pada bagian Naming (A) mendemonstrasikan penerapan prinsip Meaningful Names yang krusial untuk kejelasan kode. Transformasi variabel dari inisial ambigu n dan p menjadi username dan password mengubah kode dari sekadar kumpulan karakter menjadi deskripsi yang intention-revealing (mengungkapkan niat). Dalam konteks Defensive Programming, penggunaan nama yang eksplisit ini menghilangkan ambiguitas yang berbahaya; pengembang tidak perlu menebak apakah n berarti "name", "number", atau "null", sehingga risiko kesalahan penggunaan variabel dalam logika metode penyimpanan dapat diminimalisir secara signifikan. Namun, perlu dicatat bahwa penamaan kelas kelasPengguna sebaiknya diperbaiki menjadi KelasPengguna (PascalCase) dan nama metode penyimpanan menjadi kata kerja seperti simpanData agar sepenuhnya mematuhi konvensi Java standar.

Analisis pada bagian Formatting (B) menyoroti pentingnya Vertical Formatting dan indentasi yang disiplin sebagai alat navigasi logika. Perbandingan

antara blok kode pertama (terstruktur) dan kedua (terkompresi/satu baris) menunjukkan bahwa kode yang ditulis memanjang ke samping tanpa spasi putih (whitespace) sangat sulit untuk didebug. Struktur loop for dan percabangan if-else yang dipadatkan menyembunyikan alur kontrol program, menjadikan kode tempat persembunyian yang sempurna bagi bug. Oleh karena itu, penerapan format yang rapi seperti pada blok pertama—meskipun masih perlu penyempurnaan pada penempatan kurung kurawal—adalah bentuk pertahanan visual yang wajib dilakukan agar hierarki kode dapat dipindai dan dipahami dengan cepat oleh mata manusia.

II.2 Defensive Programming

II.2.A. Source code

A. Tugas1

```
package Tugas.DefensiveProgramming;

public class Tugas1 {
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        try {
            System.out.println(3 / 0);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic          Exception:
"+e.getMessage());
        }
    }
}
```

B. ListOfNumbers2

```
package Tugas.DefensiveProgramming;

import java.io.*;
import java.util.Vector;

public class ListOfNumbers2 {
    private Vector victor;
    private static final int size = 10;
```

```

public ListOfNumbers2() throws IOException {
    vector = new Vector(size);
    for (int i = 0; i < size; i++)
        vector.addElement(new Integer(i));
    this.readList("infile.txt");
    this.writeList();
}

public void readList(String fileName) {
    try {
        String line = null;
        RandomAccessFile raf = new RandomAccessFile(fileName,
"r");

        while ((line = raf.readLine()) != null) {
            Integer i = new Integer(Integer.parseInt(line));
            System.out.println(i);
            vector.addElement(i);
        }
    } catch (Exception e){
        System.out.println("Error in reading file:
"+e.getMessage());
    }
}

public void writeList() throws IOException {
    try {
        PrintWriter out = null;
        out = new PrintWriter(new FileWriter("outfile.txt"));

        for (int i = 0; i < vector.size(); i++) {
            out.println("Value at: " + i + " = " +
vector.elementAt(i));
            //Tambahkan Exception jika array melebihi batas
            //Tambahkan Exception untuk output error
            //Tambahkan Exception untuk exception akhir
menampilkan Closing Print Writer jika out tidak null dan Printwriter
not open jika out null

        }
    } catch (FileNotFoundException e){
        System.out.println("Error in writing file:
"+e.getMessage());
    } catch (ArrayIndexOutOfBoundsException e){

```

```

        System.out.println("Error in array out of bounds:
"+e.getMessage());
    }catch (Exception e){
        System.out.println("Error: "+e.getMessage());
    }
}
public static void main(String[] args) throws IOException {
    new ListOfNumbers2();
}
}

```

II.2.B. Analisis

Analisis pada kode Tugas1 menunjukkan penerapan dasar Defensive Programming melalui mekanisme Exception Handling yang presisi. Operasi pembagian dengan nol (3/0), yang secara normal akan menyebabkan penghentian paksa aplikasi (crash), berhasil ditangani dengan blok try-catch spesifik untuk `ArithmeticException`. Hal ini menjaga keberlangsungan program (program continuity). Namun, pada kelas `ListOfNumbers2` (metode `readList`), pertahanan tersebut menjadi kurang efektif karena penggunaan `catch (Exception e)` yang bersifat global (generic). Dalam prinsip Clean Code, menangkap semua jenis error dalam satu wadah adalah praktik buruk (bad practice) karena menyamarkan penyebab asli kesalahan—apakah karena file tidak ditemukan (`FileNotFoundException`) atau format angka yang salah (`NumberFormatException`)—sehingga mempersulit proses debugging.

Analisis pada metode `writeList` di `ListOfNumbers2` memperlihatkan struktur pertahanan yang lebih baik menggunakan Multiple Catch Blocks, di mana penanganan kesalahan diurutkan dari yang paling spesifik (`FileNotFoundException`) ke yang paling umum. Meskipun demikian, kode ini memiliki celah defensif yang kritis: ketiadaan blok `finally` atau mekanisme `try-with-resources` untuk menutup objek `PrintWriter`. Seperti yang diisyaratkan oleh komentar dalam kode, kegagalan menutup stream I/O dapat menyebabkan kebocoran sumber daya (resource leak) dan data yang tidak tersimpan ke file. Defensive programming yang utuh mewajibkan pembersihan sumber daya eksternal terjadi secara deterministik, terlepas dari apakah proses penulisan berhasil atau mengalami kegagalan.

BAB III. KESIMPULAN

Praktikum ini telah memberikan pemahaman mendalam bahwa menulis kode program bukan sekadar memastikan aplikasi berjalan (working software), tetapi juga memastikan kode tersebut mudah dipahami, dipelihara, dan tangguh. Melalui penerapan prinsip Clean Code, kita belajar mengubah kode yang ambigu dan berantakan menjadi narasi yang jelas. Teknik seperti penggunaan Meaningful Names (penamaan yang bermakna), Formatting yang konsisten, dan penghapusan komentar yang tidak perlu (Bad Comments) terbukti secara signifikan menurunkan beban kognitif saat membaca kode. Selain itu, refaktorisasi struktur logika—seperti mengganti Switch Statements yang kaku dengan Polimorfisme dan menerapkan Data Abstraction—menunjukkan bagaimana desain kode yang baik dapat meningkatkan fleksibilitas dan mempermudah pengembangan jangka panjang.

Di sisi lain, penerapan Defensive Programming mengajarkan kita untuk bersikap proaktif terhadap potensi kegagalan sistem. Melalui mekanisme Exception Handling yang terstruktur (penggunaan blok try-catch yang spesifik), kita dapat mencegah aplikasi berhenti secara mendadak saat menghadapi error. Pemahaman tentang hierarki Exception (menangkap subclass sebelum parent class) dan penggunaan blok finally untuk manajemen sumber daya adalah kunci untuk menjaga stabilitas aplikasi. Ditambah dengan penggunaan Assertion untuk memvalidasi asumsi dasar (Fail Fast), kombinasi antara kode yang bersih (clean) dan pertahanan yang berlapis (defensive) menghasilkan perangkat lunak yang tidak hanya Reliable (andal) tetapi juga Maintainable (mudah dirawat) oleh tim pengembang.